
Introduction to Large Language Models

Week 5 • Lecture 4

Tokenizers: BPE, WordPiece and SentencePiece

Detailed study notes built from the lecture slides of Prof. Mitesh M. Khapra
(AI4Bharat, IIT Madras)

What these notes cover

- **Introduction:** where the tokenizer sits in the LM pipeline, how to build a vocabulary, and the four challenges (size, OOV, typos, open vocabulary).
- **Module 4.1, Types of tokenization:** character, word and sub-word level, and the wishlist.
- **Module 4.2, Byte Pair Encoding:** pre-processing, the full algorithm and code, a complete worked example with every count, inference, non-segmented languages, and byte-level BPE.
- **Module 4.3, WordPiece:** the PMI-style score, ## prefixes, a worked score table, and longest-match inference.
- **Module 4.4, SentencePiece / Unigram LM:** probabilistic segmentation, EM, a Viterbi walkthrough on “whereby” with a lattice diagram, and Morfessor (MAP).
- **Practice:** numerical problems with worked solutions, subjective questions with model answers, and MCQs with explanations.

Contents

1	Introduction: why tokenizers matter	3
1.1	Language modelling needs a consistent split into tokens	3
1.2	Where the tokenizer sits in the pipeline	3
1.3	The first step: build (learn) the vocabulary	3
1.4	Some questions	4
1.5	Challenges in building a vocabulary	4
2	Module 4.1: Types of tokenization	5
2.1	Word-level tokenization	5
2.2	Character-level tokenization	6
2.3	Sub-word-level tokenization	6
2.4	The wishlist for a tokenizer	7
3	Module 4.2: Byte Pair Encoding (BPE)	7
3.1	General pre-processing steps	7
3.2	The BPE learning algorithm	8
3.3	Worked example from the lecture	10
3.4	After training: the final vocabulary and inference	13
3.5	BPE for non-segmented languages	14
3.6	Growing vs shrinking a vocabulary	15
4	Module 4.3: WordPiece	15
4.1	The problem with raw frequency	15
4.2	The WordPiece score	16
4.3	Sub-word prefix notation	17
4.4	Worked example: scores on the lecture corpus	17
4.5	WordPiece at inference: greedy longest-match-first	18
5	Module 4.4: SentencePiece and the Unigram language model	19
5.1	One word, many segmentations	19
5.2	The probabilistic approach	19
5.3	The Unigram language model	20
5.4	The Unigram (SentencePiece) training algorithm	21
5.5	Viterbi decoding: segmenting “whereby”	22
5.6	Morfessor	24
6	Putting it all together	25

7 Quick-revision cheat sheet	26
8 Numerical problems with worked solutions	26
9 Subjective questions with model answers	31
10 Multiple-choice questions	34

1 Introduction: why tokenizers matter

1.1 Language modelling needs a consistent split into tokens

Recall that language modelling computes probabilities over a **sequence of tokens**:

Formula : chain-rule factorisation

$$P(x_1, \dots, x_T) = P(x_1) \prod_{i=2}^T P(x_i | x_1, \dots, x_{i-1}) = P(x_1) P(x_2 | x_1) \cdots P(x_T | x_{T-1}, \dots, x_1).$$

Each x_i is an element of a finite **vocabulary** $\mathcal{V}$. The model's output layer is a softmax over $|\mathcal{V}|$ classes.

This formula silently assumes we know what a “token” x_i is. **We must split the input text into tokens in a consistent way**: the same text must always give the same token sequence, and the split must be reversible so we can turn predicted tokens back into text. The simplest approach is to split on **whitespace**.

1.2 Where the tokenizer sits in the pipeline

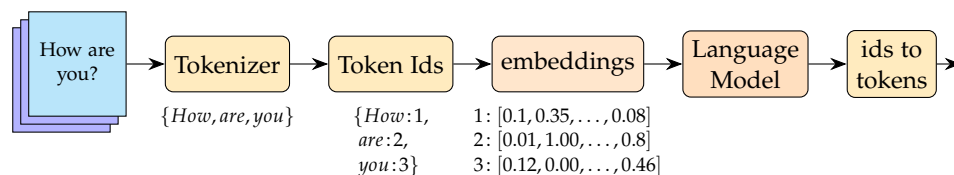


Figure 1: Text $\rightarrow$ tokens $\rightarrow$ integer ids $\rightarrow$ embedding vectors $\rightarrow$ LM $\rightarrow$ predicted ids $\rightarrow$ text.

1. The **tokenizer** splits the raw string into tokens: “How are you?” $\rightarrow$ {How, are, you}.
2. Each token is mapped to an integer **token id** through the vocabulary (a lookup table).
3. Each id selects a row of the **embedding matrix** $E \in \mathbb{R}^{|\mathcal{V}| \times d}$: id 1 $\mapsto$ [0.1, 0.35, ..., 0.08].
4. The **language model** processes the vectors and predicts ids.
5. Predicted **ids are mapped back to tokens**, and the tokens back to text (detokenisation).

Key idea

The tokenizer is **not** part of the neural network, and it is **not trained by gradient descent**. It is learned beforehand from a corpus (“training the tokenizer”) and then **frozen**. The LM is trained on its output, so changing the tokenizer later means re-training (or at least re-learning the embeddings of) the model.

1.3 The first step: build (learn) the vocabulary

The first step is to **build (learn) the vocabulary** $\mathcal{V}$ of unique tokens x_i . The fundamental question is therefore: *how do we build a vocabulary from a large corpus containing billions of sentences* (Wikipedia, books, blogs, web pages)?

A naive recipe:

1. Split the text into words on whitespace (this step is called **pre-tokenization**).
2. Add every unique word in the corpus to the vocabulary.
3. Add **special tokens** such as `<go>`, `<stop>`, `<mask>`, `<sep>`, `<cls>` and others, depending on the downstream tasks and the architecture (GPT vs BERT).

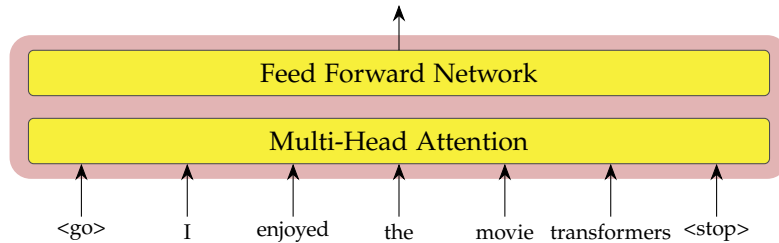


Figure 2: Special tokens such as `<go>` and `<stop>` are vocabulary entries like any word.

1.4 Some questions

1. Is splitting the input into words on whitespace a good approach?
2. If so, do we treat “*enjoy*” and “*enjoyed*” as separate tokens? They share almost all their meaning, yet a word-level vocabulary gives them unrelated ids and unrelated embeddings.
3. What about languages like **Japanese**, which do **not use word delimiters** such as spaces? For example, the Japanese for “I enjoyed watching the movie Transformers” is written as one unbroken string of characters (romanised: *eiga ‘toransufōmā’ o tanoshiku mimashita*).
4. Why not treat each individual character as the vocabulary?
5. Finally, what are good tokens?

1.5 Challenges in building a vocabulary

1. **What should the size of the vocabulary be?**
The larger the vocabulary, the larger the **embedding matrix** ($|\mathcal{V}| \times d$ parameters) and the more expensive the **softmax** over $|\mathcal{V}|$ outputs at every position. What is the optimal size?
2. **Out-of-vocabulary (OOV) words.**
If we cap the vocabulary (say at 250K, or 50K), we need a mechanism for words not in it. The classical answer, a single `<UNK>` token, throws away all information about the word.
3. **Misspelled words in the corpus.**
Corpora are often built by scraping the web, so they contain typos (“recieve”, “teh”). Such erroneous words should *not* each become a unique vocabulary entry.
4. **The open vocabulary problem.**
New words can be constructed, for example in **agglutinative languages** (Turkish, Finnish, Tamil, Kannada, Malayalam), by combining existing morphemes. Names and numbers make the vocabulary, in principle, **infinite**, which makes tasks like **machine translation** challenging.

Formula : how vocabulary size costs parameters and compute

With model width d and vocabulary $\mathcal{V}$:

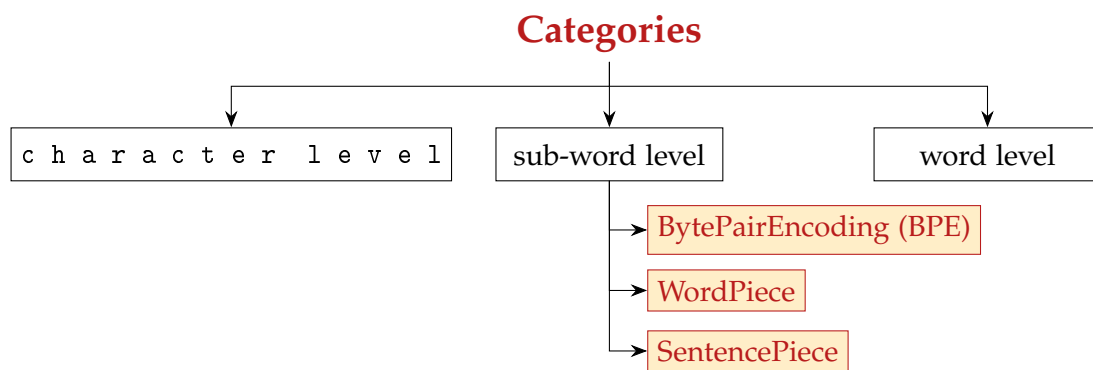
$$\begin{aligned}\#params_{\text{input emb}} &= |\mathcal{V}| \cdot d, & \#params_{\text{output layer}} &= |\mathcal{V}| \cdot d (+|\mathcal{V}| \text{ bias}), \\ \text{FLOPs}_{\text{softmax logits}} &\approx 2 |\mathcal{V}| d \text{ per position.}\end{aligned}$$

Example: $|\mathcal{V}| = 50,000$, $d = 768$ gives 38.4M parameters for the input embedding alone. A 250K word-level vocabulary with $d = 4096$ gives 1.02B parameters per embedding matrix.

Intuition : the fundamental trade-off

There are two extremes. **Big units** (whole words) give short sequences but a huge vocabulary, many OOVs and no sharing between related words. **Tiny units** (characters) give a tiny vocabulary and no OOVs, but very long sequences, and each unit carries almost no meaning, so the model must spend its capacity assembling words from letters. Good tokenization finds units in between: frequent enough to be learned well, meaningful enough to be useful.

2 Module 4.1: Types of tokenization



* There are many categories; these three sub-word methods are the most commonly used.

Figure 3: The three granularities of tokenization, and the three sub-word algorithms this week covers.

We compare the three granularities on the running example “*Hmm.. I know, I don’t know*”.

2.1 Word-level tokenization

Split on whitespace (and optionally punctuation). Each unique word is a vocabulary entry.

Worked example : word level

Whitespace split: [Hmm., I, know,, I, don’t, know], so **6 tokens**. The vocabulary here is {Hmm., I, know,, don’t, know}, so $|\mathcal{V}| = 5$. Note that know, and know become *different* tokens, which is why real word tokenizers also split off punctuation.

- + Short sequences, and each token is a meaningful unit.
- Huge vocabularies (English web text has millions of unique “words”), so a large embedding matrix and an expensive softmax.

- OOV problem: any unseen word (new name, typo, inflection) becomes $\langle \text{UNK} \rangle$.
- No sharing: *enjoy*, *enjoyed* and *enjoying* get unrelated embeddings, and rare forms are poorly trained.
- Needs word boundaries, so it fails for Japanese, Chinese and Thai.

2.2 Character-level tokenization

Every character (including space and punctuation) is a token.

Worked example : character level

“Hmm.. I know, I don’t know” has **26 characters, hence 26 tokens**. Its vocabulary is the set of distinct characters $\{\text{H, m, ., , , I, k, n, o, w, , , d, ', t}\}$, so $|\mathcal{V}| = 13$.

- + Tiny vocabulary (a few dozen to a few hundred symbols for most scripts), so embedding and softmax costs are negligible.
- + Essentially no OOV: any word in the script can be spelled out.
- + Robust to typos (“recieve” is only one character away from “receive”).
- Very long sequences ($\approx 4\text{--}5\times$ longer than words in English). Self-attention costs $O(T^2)$, so that is $\approx 16\text{--}25\times$ more attention compute, and the context window fills up quickly.
- Individual characters carry little meaning, so the model has to spend depth composing letters into words.

2.3 Sub-word-level tokenization

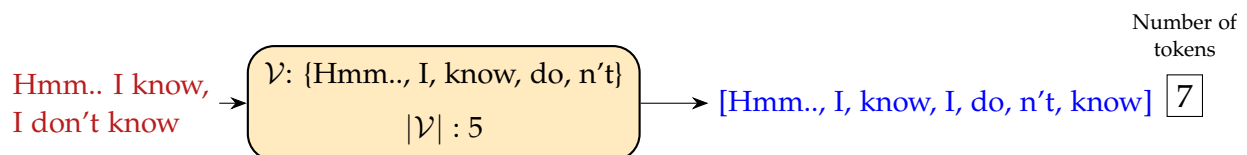


Figure 4: Sub-word tokenization of the running example: 7 tokens with a 5-entry vocabulary.

- A **middle ground** between character-level and word-level tokenizers.
- The vocabulary is carefully built based on the **frequency of occurrence of subwords**.
- The **most frequently occurring words are preserved as is**, and **rare words are split into subword units**.
- The size of the vocabulary is **moderate**.
- Example: the tokenizer breaks “don’t” into “do” and “n’t” by learning that “n’t” occurs frequently in the corpus (don’t, can’t, isn’t, wasn’t, ...). We then have a representation for “do” and a representation for “n’t”, and “n’t” can be *reused* for every negated verb.
- Therefore sub-word tokenizers are **preferred for LLMs**.

Granularity	Tokens for the example	$ \mathcal{V} $ (example)	Typical real $ \mathcal{V} $
Character	26	13	10^2 (or 256 bytes)
Sub-word	7	5	30K–250K
Word	6	5	10^5 – 10^6 + (and still OOVs)

Intuition : sub-words as a compression code

Sub-word tokenization works like a data-compression dictionary. Frequent strings (“the”, “ing”, “tion”) get their own short codes, and rare strings are spelled out from smaller pieces. That is literally where BPE comes from: it was first a 1994 compression algorithm (Gage) and was adapted to NLP by Sennrich et al. (2016). It gives short sequences for common text and still covers everything, which is the best of both worlds.

2.4 The wishlist for a tokenizer

Key idea : what we want

1. **Moderate-sized vocabulary.**
2. **Efficiently handle unknown words during inference** (no information-destroying <UNK>).
3. **Be language agnostic** (work even without whitespace, and across scripts).

Keep this list in mind. BPE ticks (1) and (2). WordPiece refines *how* merges are chosen. SentencePiece (with the unigram model) ticks (3), because it does not require pre-tokenized words.

Systems / engineering note : tokens are the unit of cost

Everything in LLM serving is priced and scheduled in tokens: context windows, KV-cache size ($\propto$ tokens $\times$ layers $\times d$), throughput (tokens/s) and API billing. A tokenizer that needs $1.5\times$ more tokens for a language (common for Indic languages with English-centric tokenizers) makes that language $1.5\times$ more expensive and $1.5\times$ slower, and shrinks its effective context. Tokenizer “fertility” (tokens per word) is a real fairness and efficiency metric.

3 Module 4.2: Byte Pair Encoding (BPE)

3.1 General pre-processing steps

Every sub-word scheme follows a sequence of steps to learn its vocabulary. The input corpus is often built by scraping web pages and e-books.

1. **Normalization.** Clean the text: lower-casing (“treating cases”), removing accents (café $\rightarrow$ cafe), collapsing multiple whitespaces, stripping HTML tags, Unicode normalization (NFC/NFKC), and so on.
2. **Pre-tokenization.** Split the text into words, typically on whitespace (and punctuation). Traditionally this *was* called tokenization. When it is followed by a sub-word algorithm, it is called **pre-tokenization**.
3. **Learn the vocabulary (called training)** from these words using the sub-word algorithm.

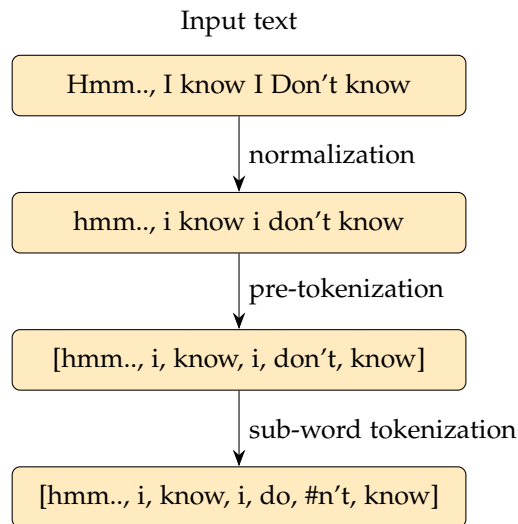


Figure 5: The three stages: normalization → pre-tokenization → sub-word tokenization.

Common pitfall / exam trap

Normalization choices are baked in *forever*. If you lower-case during tokenizer training (as in bert-base-uncased), the model can never distinguish “Apple” (the company) from “apple” (the fruit). Removing accents destroys meaning in many languages (Hindi matras, Vietnamese tones). Modern LLM tokenizers normalize very lightly.

Training vs. inference pipeline

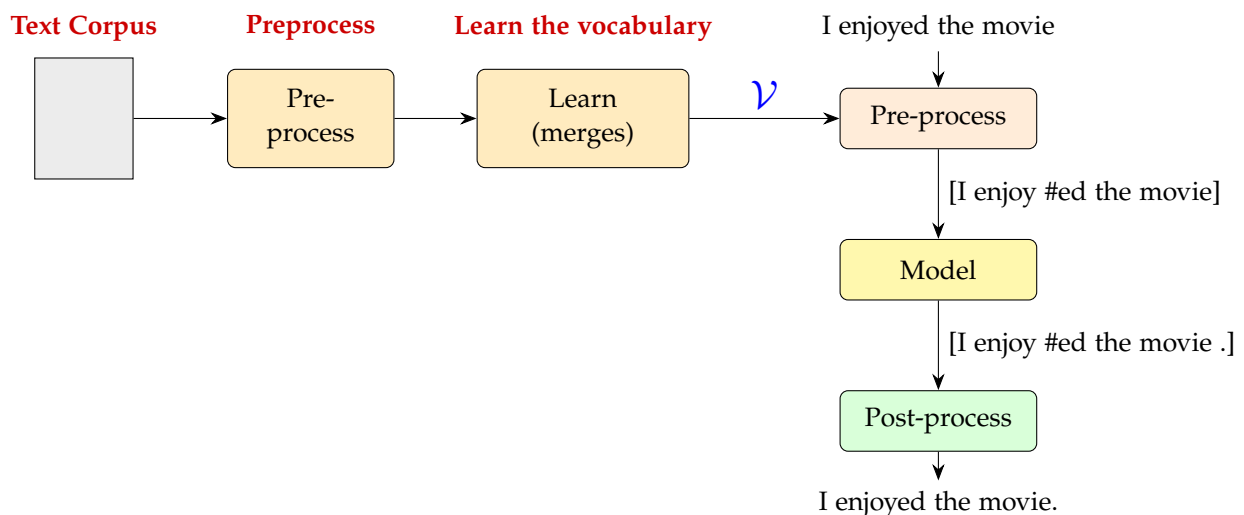


Figure 6: Left: the tokenizer is learned once from the corpus. Right: at inference, the learned $\mathcal{V}$ tokenizes the input, the model runs, and post-processing merges the sub-words back into text.

3.2 The BPE learning algorithm

1. Start with a dictionary of **words and their counts** (from pre-tokenization).
2. Append a special symbol </w> to the end of each word. It marks the **word boundary**.

3. Set the **required number of merges**. This is a hyperparameter, and it decides the final vocabulary size.
4. Initialize the **character-frequency table**, the base vocabulary: every word is split into characters.
5. Get the **frequency count for each adjacent pair** of symbols.
6. **Merge the pair with maximum occurrence** into a new symbol, add it to the vocabulary, and update the words.
7. Repeat steps 5–6 until the number of merges is reached.

The reference implementation from the slides (after Sennrich et al., 2016):

```

1 import re, collections
2
3 def get_stats(vocab):
4     pairs = collections.defaultdict(int)
5     for word, freq in vocab.items():
6         symbols = word.split()
7         for i in range(len(symbols)-1):
8             pairs[symbols[i], symbols[i+1]] += freq # each occurrence counts 'freq' times
9     return pairs
10
11 def merge_vocab(pair, v_in):
12     v_out = {}
13     bigram = re.escape(' '.join(pair))
14     p = re.compile(r'(?!\S)' + bigram + r'(?!\S)') # match the pair only as whole symbols
15     for word in v_in:
16         w_out = p.sub(' '.join(pair), word) # 'i n' -> 'in'
17         v_out[w_out] = v_in[word]
18     return v_out
19
20 vocab = {'l o w </w>': 5, 'l o w e r </w>': 2,
21         'n e w e s t </w>': 6, 'w i d e s t </w>': 3}
22 num_merges = 10
23 for i in range(num_merges):
24     pairs = get_stats(vocab)
25     best = max(pairs, key=pairs.get) # most frequent pair
26     vocab = merge_vocab(best, vocab)
27     print(best)

```

Reading the code.

- Words are stored as *space-separated symbols* ('l o w </w>'), and the dictionary maps each to its corpus count. We never iterate over the raw corpus again, only over *unique words*. That is the main efficiency trick.
- `get_stats` counts every adjacent pair *weighted by the word's frequency*.
- `merge_vocab` rewrites each word, fusing every occurrence of the chosen pair. The look-arounds `(?!\\S)` and `(?!\\S)` ensure we only fuse complete symbols (e.g. merging `e s` must not touch the `e` inside `es` of another symbol).
- The sequence of best pairs, in order, *is* the learned tokenizer: the **merge list**.

Worked example : the code's toy corpus {low:5, lower:2, newest:6, widest:3}

Pair counts at start: (e,s)= 6 + 3 = 9, (s,t)= 9, (t,</w>)= 9, (l,o)= 7, (o,w)= 7, (w,e)= 2 + 6 = 8, (n,e)= 6, (e,w)= 6, ... Merge 1: (e,s) → es (9; the first of the three tied pairs). Merge 2: (es,t) → est (9). Merge 3: (est,</w>) → est</w> (9). Merge 4: (l,o) → lo (7). Merge 5: (lo,w) → low (7). Then (n,e), (ne,w), (new,est</w>), (low,</w>), (w,i), ... Notice how the suffix est</w> ("superlative at end of word") emerges first, because it is shared across words. BPE discovers morphology from frequency alone.

3.3 Worked example from the lecture

Corpus: "knowing the name of something is different from knowing something. knowing something about everything isn't bad"

Objective: repeatedly find the most frequently occurring byte-pair. The </w> symbol identifies the word boundary.

Step 1: word counts

Word	Freq.
k n o w i n g </w>	3
t h e </w>	1
n a m e </w>	1
o f </w>	1
s o m e t h i n g </w>	2
i s </w>	1
d i f f e r e n t </w>	1
f r o m </w>	1
s o m e t h i n g . </w>	1
a b o u t </w>	1
e v e r y t h i n g </w>	1
i s n ' t </w>	1
b a d </w>	1

13 unique words ("vocab size: 13" on the slides) and 16 word occurrences in total. Note that "something." (with the full stop) is a *different* word from "something", because pre-tokenization here splits only on whitespace. Real tokenizers usually split punctuation off.

Step 2: the initial character (token) counts

We do not rescan the corpus. A character's count is the sum, over words containing it, of (occurrences in word) × (word frequency).

- 'k': only "knowing" contains k, once, and knowing occurs 3 times, so $1 \times 3 = 3$.
- 'n': knowing has 2 n's ($2 \times 3 = 6$), name $1 \times 1 = 1$, something $1 \times 2 = 2$, different 1, something. 1, everything 1, isn't 1. Total $6 + 1 + 2 + 1 + 1 + 1 + 1 = 13$.
- 'o': knowing 3, of 1, something 2, from 1, something. 1, about 1. Total 9.
- '</w>': one per word occurrence, $3 + 1 + 1 + 1 + 2 + \dots + 1 = 16$.

Char	Freq	Char	Freq	Char	Freq	Char	Freq
</w>	16	t	8	f	4	d	2
n	13	g	7	k	3	b	2
i	10	h	5	w	3	.	1
o	9	m	5	a	3	u	1
e	9	s	5	r	3	v	1
						y	1
						'	1

Initial vocabulary size: 22 (21 characters + </w>). The counts add up to 112 symbols, the corpus length in the initial segmentation.

Step 3: byte-pair counts

Walk through every word and count each adjacent pair, weighted by frequency.

- From “k n o w i n g </w>” (freq 3): (k,n), (n,o), (o,w), (w,i), (i,n), (n,g), (g,</w>), each +3.
- (i,n) also appears in something (+2), something. (+1) and everything (+1), so (i,n)= 3 + 2 + 1 + 1 = 7.
- (n,g) appears in exactly the same four words, so (n,g)= 7 as well: a **tie**.

Pair	#	Pair	#	Pair	#	Pair	#
(i,n)	7	(n,o)	3	(i,s)	2	(d,</w>)	1
(n,g)	7	(o,w)	3	('t)	1	(d,i)	1
(g,</w>)	6	(s,o)	3	(.,</w>)	1	(e,n)	1
(t,h)	5	(t,</w>)	3	(a,b)	1	(e,v)	1
(h,i)	4	(w,i)	3	(a,d)	1	(f,</w>)	1
(m,e)	4	(e,</w>)	2	(a,m)	1	(f,e)	1
(o,m)	4	(e,r)	2	(b,a)	1	(f,f)	1
(e,t)	3			(b,o)	1	(f,r)	1
(k,n)	3					(g,.)	1
						(h,e)	1
						(i,f)	1
						(m,</w>)	1
						(n,')	1
						(n,a)	1

There are 49 distinct pairs initially.

Step 4: merge the most frequent pair, (i, n) → in

The two tied pairs are (i,n) and (n,g). The slides (and Python’s max, which returns the first maximum it finds) pick **(i,n)**. Merge it everywhere:

k n o w **in** g </w> (3) s o m e t h **in** g </w> (2)
s o m e t h **in** g . </w> (1) e v e r y t h **in** g </w> (1)

Update the token counts. The new token in gets count 7. Each of its 7 occurrences consumed one ‘i’ and one ‘n’, so

$$'n' : 13 - 7 = 6, \quad 'i' : 10 - 7 = 3, \quad 'in' : 7.$$

The vocabulary grows from 22 to 23 (the new token in is *added*; ‘i’ and ‘n’ stay in the vocabulary because they still occur, and even if they did not, BPE keeps the base alphabet so any string stays encodable).

Common pitfall / exam trap : tie-breaking

BPE by itself has no principled rule for ties such as $(i,n)=(n,g)=7$. Implementations break ties by order of discovery or lexicographically, and *different tie-breaks give different tokenizers*. This weakness is exactly what motivates WordPiece’s score (Section 4).

Step 5: now treat `in` as a single symbol and repeat

Because `in` is now one symbol, the old pairs that involved ‘i’ or ‘n’ there disappear and new pairs appear:

- $(w,i) \rightarrow (w,in)$: 3 (from knowing); (i,n) disappears; $(n,g) \rightarrow (in,g)$: 7.
- $(h,i) \rightarrow (h,in)$: $2 + 1 + 1 = 4$ (something, something., everything).

The new byte pairs are therefore $(w,in):3$, $(in,g):7$, $(h,in):4$. The most frequent is $(in,g)=7$, so we merge `in + g` → `ing` (count 7). Now treat `ing` as a single token and repeat.

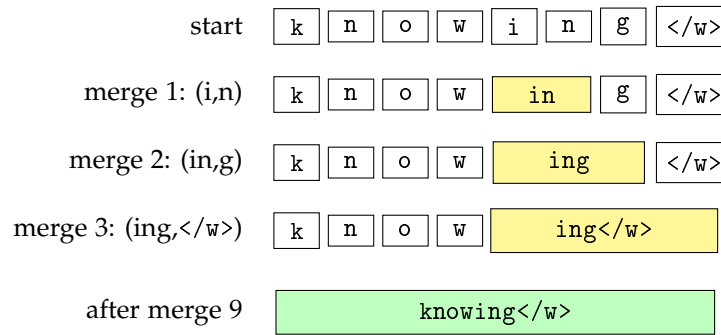


Figure 7: How “knowing” is progressively merged (continuing the lecture’s run with the same tie-breaking).

Continuing: the first 12 merges

Running the algorithm to completion (verified in code) gives:

#	Merge	Count	New token	$ \mathcal{V} $	Corpus length (symbols)
0	(start)	—	—	22	112
1	(i, n)	7	in	23	105
2	(in, g)	7	ing	24	98
3	(ing, </w>)	6	ing</w>	25	92
4	(t, h)	5	th	26	87
5	(m, e)	4	me	27	83
6	(k, n)	3	kn	28	80
7	(kn, o)	3	kno	29	77
8	(kno, w)	3	know	30	74
9	(know, ing</w>)	3	knowing</w>	31	71
10	(s, o)	3	so	32	68
11	(so, me)	3	some	33	65
12	(some, th)	3	someth	34	62

Each merge with count c **shrinks the corpus by exactly c symbols** (every occurrence turns two

symbols into one). For example $112 - 7 = 105$ and $105 - 7 = 98$. This is BPE's compression view: every merge is the currently best single step for shortening the corpus.

Key idea : vocabulary size is a knob

$$|\mathcal{V}_{\text{final}}| = |\mathcal{V}_{\text{base}}| + \text{\#merges} \quad (\text{here } 22 + N).$$

The number of merges is the hyperparameter that sets the vocabulary size. On this toy corpus, 53 merges turn every word into a single token. The slides stop at 45 merges, where some words are still split. For a large corpus, we typically end up with a vocabulary *smaller* than taking every word as a token (30K–50K vs millions of distinct words).

3.4 After training: the final vocabulary and inference

- The final vocabulary contains the **initial vocabulary and all the merges, in order**. The order matters.
- Frequent words end up as single tokens. **Rare words are broken into two or more subwords**.
- **At test time**, an input word is split into a sequence of characters, and the characters are merged into larger, known symbols by **replaying the learned merges in the order they were learned**.

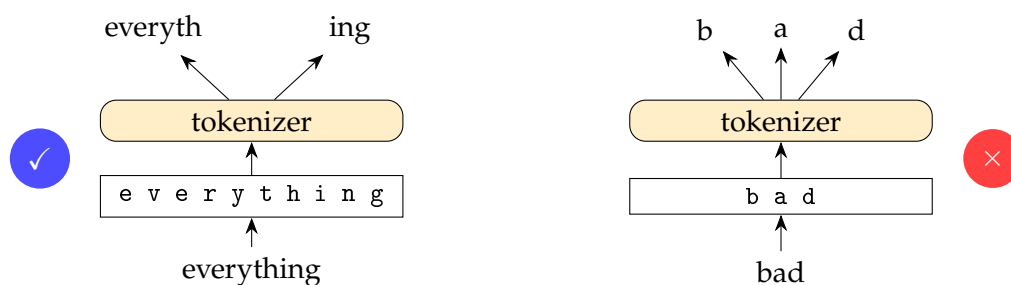


Figure 8: At test time, “everything” becomes *everyth* + *ing* (useful sub-words). A word whose pairs were never merged, like “bad” here, falls back to single characters. It is still encodable (no <UNK>), but inefficiently.

- The pair (i,n) is merged first, followed by (in,g), exactly the order learned during training.
- The algorithm lets us **adjust the vocabulary size as a function of the number of merges**.

Encoding algorithm (inference)

```

1 def bpe_encode(word, merges): # merges: list of pairs in learned order
2     symbols = list(word) + ['</w>']
3     for a, b in merges: # replay merges by rank
4         i, out = 0, []
5         while i < len(symbols):
6             if i < len(symbols)-1 and symbols[i] == a and symbols[i+1] == b:
7                 out.append(a + b); i += 2 # fuse this occurrence
8             else:
9                 out.append(symbols[i]); i += 1
10        symbols = out
11    return symbols

```

Worked example : tokenize “hello lol” with $\mathcal{V} = \{a, b, c, \dots, z, lo, he\}$ (merge order: lo, then he)

1. Pre-tokenize and split into characters: [h e l l o], [l o l].
2. Search for the byte-pair ‘lo’. If present, merge. Yes, so: [h e l lo], [lo l].
3. Search for the byte-pair ‘he’. If present, merge. Yes, so: [he l lo], [lo l].
4. Return the text after merging: **he #l #lo, lo #l**.

Here “#” marks a token that *continues* a word (it is not the start of a word). This is how the model later knows where the word boundaries were, so that post-processing can glue “he #l #lo” back into “hello”.

Intuition : why replay merges in order, rather than “longest match”?

The merge list encodes *how the training corpus was compressed*. Replaying it in order reproduces exactly the segmentation that the model saw during training for known words, which makes tokenization consistent between training and inference. A greedy longest-match over the vocabulary can produce segmentations the model never saw. (WordPiece inference does use longest match, but with its own vocabulary construction.)

Systems / engineering note : complexity and fast implementations

The naive loop re-counts all pairs after every merge, which costs $O(\text{\#merges} \times \text{corpus symbols})$. Practical trainers (HuggingFace tokenizers, SentencePiece) keep pair counts in a priority queue and update *only the pairs adjacent to the merged positions*, using an index from pairs to words. Encoding at inference is typically done by repeatedly merging the lowest-rank adjacent pair (rank = merge order) with a heap, which is $O(n \log n)$ per word, plus a cache for frequent words. OpenAI’s tiktoken implements this in Rust.

3.5 BPE for non-segmented languages

- Languages such as **Japanese** and **Korean** (and Chinese, Thai) are **non-segmented**: they do not put spaces between words.
- But **BPE requires space-separated words** (pre-tokenization produces the word-count dictionary).
- In practice we can use **language-specific**, morphology-based word segmenters such as **Juman++** or **MeCab** for Japanese to pre-tokenize, and then run BPE.
- However, for **multilingual** translation, having a **language-agnostic** tokenizer is desirable. One segmenter per language does not scale, and code-mixed text breaks it. This motivates SentencePiece ([Section 5](#)).

Beyond the slides : byte-level BPE (GPT-2 onwards)

If the base vocabulary is Unicode *characters*, a multilingual corpus can have 100K+ distinct characters (Chinese alone has tens of thousands). GPT-2 instead runs BPE over **UTF-8 bytes**: the base vocabulary is exactly **256 byte values**, so *any* string in any language (and emoji, code, binary junk) is encodable with zero OOV, and merges build up characters and then words. GPT-2 used 50,257 tokens (256 bytes + 50,000 merges + <|endoftext|>). A regex pre-tokenizer stops merges from crossing categories (letters vs digits vs punctuation), so “dog.” and “dog!” share the token “dog”. Modern tokenizers (c1100k, Llama 3’s 128K vocabulary) follow this

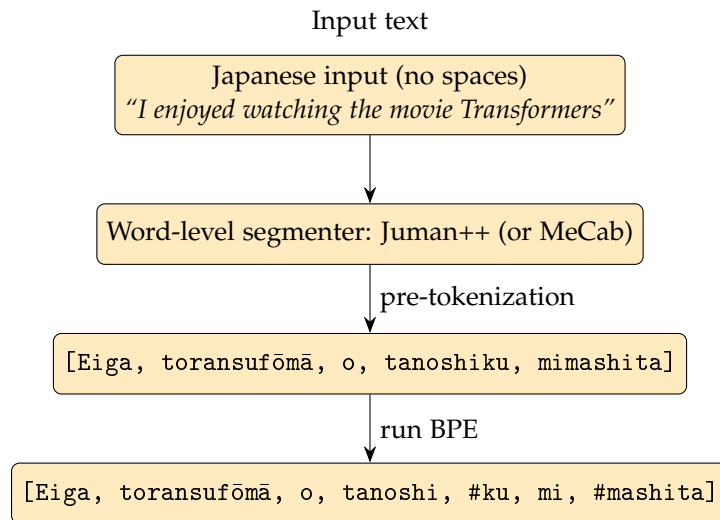
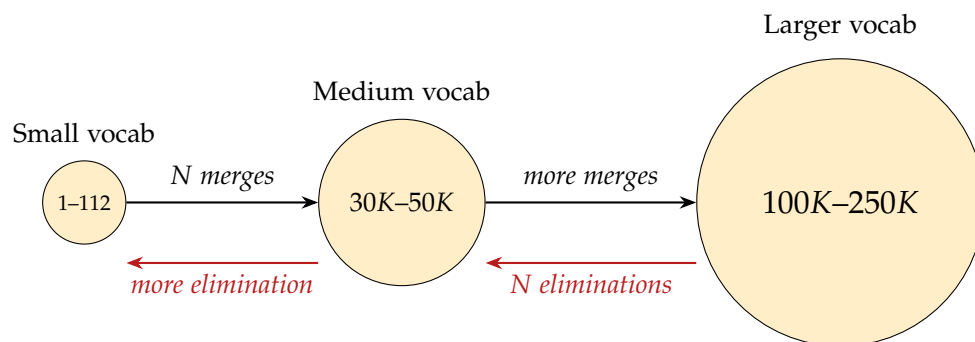


Figure 9: BPE for Japanese needs a language-specific segmenter first. (Shown in romaji; the original is in Japanese script.)

recipe. This is also why BPE is called “byte pair” encoding: in Gage’s original compression algorithm the units really were bytes.

3.6 Growing vs shrinking a vocabulary



BPE / WordPiece grow (black arrows): start from characters, keep merging.

Unigram / SentencePiece shrinks (red arrows): start large, keep eliminating.

Figure 10: Two directions for reaching a target vocabulary size.

BPE (and WordPiece, next) **start with a character-level vocabulary and keep merging until a desired vocabulary size is reached**. We can also do the reverse: **start with a (large) word-level vocabulary and keep eliminating until the desired size is reached**. That is the unigram approach used in SentencePiece (Section 5).

4 Module 4.3: WordPiece

4.1 The problem with raw frequency

In BPE we merged the pair of tokens with the **highest frequency of occurrence**. What if more than one pair has the same frequency, for example (i,n) and (n,g), both 7? And more fundamentally: is

raw frequency even the right criterion? A pair of two *very common* symbols will co-occur often just by chance.

Pair	Frequency
(k, n)	3
(n, o)	3
(o, w)	3
(w, i)	3
(i, n)	7
(n, g)	7
(g, .)	1
(t, h)	5
(h, e)	1
(e, </w>)	2
⋮	⋮
(a, d)	1

4.2 The WordPiece score

WordPiece takes the **frequency of the individual tokens in the pair** into account:

Formula : WordPiece merge score

$$\text{score}(\alpha, \beta) = \frac{\text{count}(\alpha, \beta)}{\text{count}(\alpha) \text{count}(\beta)}$$

At each step, **merge the pair with the highest score**. The numerator is how often α is immediately followed by β . The denominator is the product of the individual counts.

Now we can select a pair of tokens whose individual tokens are **less frequent in the vocabulary**: pairs that stick together *more than chance would predict*.

Intuition : WordPiece score = pointwise mutual information (PMI)

Let N be the total number of symbols. Then $P(\alpha) \approx \frac{c(\alpha)}{N}$ and $P(\alpha\beta) \approx \frac{c(\alpha, \beta)}{N}$, so

$$\frac{P(\alpha\beta)}{P(\alpha)P(\beta)} \approx \frac{c(\alpha, \beta)/N}{c(\alpha)c(\beta)/N^2} = N \cdot \text{score}(\alpha, \beta).$$

Since N is the same for every pair, ranking by score is the same as ranking by $\text{PMI} = \log \frac{P(\alpha\beta)}{P(\alpha)P(\beta)}$. $\text{PMI} > 0$ means “these two co-occur more than if they were independent”. Two very frequent symbols (like ‘n’ and ‘o’) co-occur often by chance alone, so their PMI is low. Two symbols that almost *only* appear together (like ‘q’ and ‘u’ in English) have high PMI and are good candidates for a single unit.

Beyond the slides : derivation, the likelihood view (why Google designed it this way)

WordPiece (Schuster & Nakajima, 2012, used for Japanese/Korean voice search and later in BERT) picks the merge that *most increases the likelihood of the training data* under a unigram model of the current tokens. Suppose that merging α, β into $\alpha\beta$ replaces $c_{\alpha\beta}$ adjacent occurrences. Under a unigram LM with maximum-likelihood probabilities, the log-likelihood contribution of those positions changes from $c_{\alpha\beta} [\log P(\alpha) + \log P(\beta)]$ to $c_{\alpha\beta} \log P(\alpha\beta)$. Ignoring the small

change in N and in the other counts, the gain is

$$\Delta \log L \approx c_{\alpha\beta} \log \frac{P(\alpha\beta)}{P(\alpha)P(\beta)}.$$

The lecture's score keeps the ratio inside the log (without the extra $c_{\alpha\beta}$ weight). It is the form used by the HuggingFace WordPiece trainer and the one you should use in exams.

4.3 Sub-word prefix notation

WordPiece does *not* use an end-of-word marker. Instead, every sub-word that **continues** a word is prefixed with `#` (the slides use a single `#` for illustration):

knowing $\rightarrow$ k #n #o #w #i #n #g

So the initial vocabulary contains *two versions* of most characters: a word-initial one (`n`) and a continuation one (`#n`). Then “the `n` in *name*” and “the `n` in *knowing*” are different symbols.

Word	Freq	Char	Freq
k n o w i n g	3	k	3
t h e	1	#n	13
n a m e	1	#o	9
o f	1	:	:
s o m e t h i n g	2	#h	5
:	:	'	1

Note on the slides : counting convention

To keep the example comparable with BPE, the slides **ignore the prefix #** when computing counts, so ‘`n`’ and ‘`#n`’ are pooled into 13, and so on. We follow that convention below. (A strict implementation would count `n` and `#n` separately: word-initial ‘`n`’ occurs only in *name*, count 1, and `#n` has count 12. The mechanics are identical.)

4.4 Worked example: scores on the lecture corpus

Pair	count(α, β)	count(α)	count(β)	Score (exact)	Slides
(k, n)	3	k: 3	n: 13	$3/39 = 0.0769$	0.076
(n, o)	3	n: 13	o: 9	$3/117 = 0.0256$	0.02
(o, w)	3	o: 9	w: 3	$3/27 = 0.1111$	0.111
(i, n)	7	i: 10	n: 13	$7/130 = 0.0538$	0.05
(n, g)	7	n: 13	g: 7	$7/91 = 0.0769$	0.076
(t, h)	5	t: 8	h: 5	$5/40 = 0.125$	0.125
(a, d)	1	a: 3	d: 2	$1/6 = 0.1667$	0.16

Now merging is based on the score of each byte pair. Merge the pair with the highest score. Among the rows shown, that is **(a,d)** with 0.167, even though (a,d) occurs only *once*. Both ‘`a`’ and ‘`d`’ are rare, and when ‘`a`’ appears it is followed by ‘`d`’ a third of the time.

The tie is resolved. BPE could not choose between (i,n) and (n,g), both 7. WordPiece gives (n,g) = 0.0769 and (i,n) = 0.0538, so it prefers **(n,g)**. The reason is that 'g' (count 7) is much rarer than 'i' (count 10), and *every* 'g' in this corpus follows an 'n'.

Note on the slides : the full table

The slide shows only some rows. Scoring *all* pairs of this corpus (prefix ignored, no $\langle/w\rangle$) gives the true maximum $(r,y) = 1/(3 \cdot 1) = 0.333$, from “everyrything”. ‘y’ occurs once, and it follows ‘r’. Next come (a,b), (b,a) and (a,d) at 0.167, then (g,.) at $1/7 = 0.143$, then (t,h), (u,t), (y,t) and (',t) at 0.125. So on this tiny corpus the very first WordPiece merge would be ry. It also shows WordPiece’s tendency to glue *rare* symbols first.

Intuition : BPE vs WordPiece in one sentence

BPE asks “*which pair is most common?*”, and it favours pairs of frequent symbols. WordPiece asks “*which pair is most surprisingly common, given how common its parts are?*”, and it favours pairs whose parts rarely appear apart. BPE maximises compression greedily. WordPiece maximises likelihood (PMI) greedily.

4.5 WordPiece at inference: greedy longest-match-first

Beyond the slides : how BERT’s tokenizer actually encodes a word

WordPiece stores only the *final vocabulary*, not a merge list. Encoding a word uses **greedy longest-match-first** (MaxMatch):

1. Starting at the beginning of the word, find the *longest* prefix that is in the vocabulary. Emit it.
2. Continue from where it ended, now looking up ##-prefixed pieces. Emit the longest match.
3. If at some point no piece matches, the *whole word* becomes [UNK].

Classic example from the BERT/Google papers: *unaffable* → un ##aff ##able. With BERT’s 30,522-token vocab, *playing* → playing, but a rare word such as *tokenization* → token ##ization. [UNK] is rare in practice because all single characters (and their ## versions) are in the vocabulary.

	BPE	WordPiece
Merge criterion	$\max \text{count}(\alpha, \beta)$	$\max \frac{\text{count}(\alpha, \beta)}{\text{count}(\alpha)\text{count}(\beta)}$
Interpretation	Greedy compression	Greedy likelihood gain (PMI)
Ties	Frequent; broken arbitrarily	Much rarer
Word-boundary marker	End-of-word $\langle/w\rangle$ (or a leading-space marker)	Continuation prefix ##
What is stored	Ordered merge list + vocab	Vocabulary only
Inference	Replay merges in order	Greedy longest-match-first
Direction	Grow from characters	Grow from characters
Used by	GPT-2/3/4, RoBERTa, Llama 3 (byte-level)	BERT, DistilBERT, ELECTRA

Summary of Module 4.3

- WordPiece = BPE with a different merge criterion: $\text{score} = \frac{c(\alpha, \beta)}{c(\alpha)c(\beta)}$.
- It favours pairs whose individual tokens are rare (high PMI) and it resolves BPE's frequency ties.
- Continuation sub-words carry the ## prefix. Merging still grows the vocabulary from characters to a target size.

5 Module 4.4: SentencePiece and the Unigram language model

5.1 One word, many segmentations

A given word can be split into subwords in many ways. For instance, the word $X = \text{"hello"}$ can be segmented in several ways *with the same vocabulary*

$$\mathcal{V} = \{h, e, l, o, he, el, lo, ll, hell\} :$$

$$\mathbf{x}_1 = he, ll, o \quad \mathbf{x}_2 = h, el, lo \quad \mathbf{x}_3 = he, l, lo \quad \mathbf{x}_4 = hell, o$$

In fact there are **9** valid segmentations in total (listed in the practice problems).

BPE is greedy and deterministic. BPE does not choose among these. It applies its merge rules in order and outputs exactly one of them:

- With merge order $\mathcal{V} = \{h, e, l, l, o, \mathbf{he}, el, lo, ll, hell\}$ (i.e. **he** learned before **el**): $h\ e\ l\ l\ o \xrightarrow{he} he\ l\ l\ o \xrightarrow{el} (no\ 'e'\ left) \xrightarrow{lo} he\ l\ lo \xrightarrow{ll} (no\ 'l\ l'\ adjacent\ any\ more) \Rightarrow \mathbf{he, l, lo}$.
- With merge order $\mathcal{V} = \{h, e, l, l, o, \mathbf{el}, he, lo, ll, hell\}$: $h\ e\ l\ l\ o \xrightarrow{el} h\ e\ l\ lo \xrightarrow{he} (no\ 'h\ e'\ adjacent) \xrightarrow{lo} h\ el\ lo \Rightarrow \mathbf{h, el, lo}$.

Same vocabulary, different merge order, different output. And neither output was chosen because it is “best”. We therefore say **BPE is greedy and deterministic**. (**BPE-dropout**, Provilkov et al. 2020, randomly skips merges during training to make it stochastic; this acts as a regulariser.)

5.2 The probabilistic approach

Instead of rules, **find the subword sequence $\mathbf{x}^* \in \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_k\}$ that maximizes the likelihood of the word X .**



Figure 11: The word is observed. Its segmentation into sub-words is a hidden (latent) variable.

- The “word” X in SentencePiece means a **sequence of characters or words, without spaces**. SentencePiece treats the input as a raw character stream and turns the space into an ordinary symbol. The library uses the character U+2581 (“lower one-eighth block”), which we print here as `_`. So “Hello world” $\rightarrow$ `_Hello _world`.

- Therefore it can be applied to languages like **Chinese and Japanese**, which do not use any word delimiters. No language-specific pre-tokenizer is needed. This ticks the “language agnostic” item of the wishlist.
- Because the space is kept as a symbol, detokenization is lossless: just concatenate the pieces and replace _ with a space.

5.3 The Unigram language model

Formula : unigram LM over sub-words

Let $\mathbf{x} = (x_1, x_2, \dots, x_n)$ be a sub-word sequence of length n . Under a unigram LM each sub-word is drawn independently:

$$P(\mathbf{x}) = \prod_{i=1}^n P(x_i), \quad \sum_{x \in \mathcal{V}} p(x) = 1.$$

The objective is to find, among all segmentation candidates $S(X)$ of the input X , the one that maximizes the (log-)likelihood:

$$\mathbf{x}^* = \arg \max_{\mathbf{x} \in S(X)} P(\mathbf{x}) = \arg \max_{\mathbf{x} \in S(X)} \sum_{i=1}^n \log P(x_i).$$

We use **Viterbi decoding** to find $\mathbf{x}^*$ efficiently (below). For the *whole dataset* D we define the likelihood

Formula : corpus likelihood with latent segmentations

$$\mathcal{L} = \sum_{s=1}^{|D|} \log (P(X^s)) = \sum_{s=1}^{|D|} \log \left(\sum_{\mathbf{x} \in S(X^s)} P(\mathbf{x}) \right).$$

The probability of a word *marginalises over all its segmentations*. Recall that the sub-word probabilities $p(x_i)$ are tied to **hidden (latent)** segmentation choices. Therefore, given the vocabulary $\mathcal{V}$, the **Expectation–Maximization (EM)** algorithm is used to maximize the likelihood, just as for mixture models or HMMs, where we also have “a sum inside the log”.

Intuition : why EM?

If we *knew* the true segmentation of every word, estimating $p(x)$ would be trivial: count each sub-word and normalise. If we *knew* $p(x)$, finding the best segmentation would be easy (Viterbi). We know neither, so we alternate: guess probabilities, segment the corpus, re-count, re-estimate, and repeat. Each round cannot decrease the likelihood.

Worked example: segmenting “knowing”

Worked example : from the slides

Let $X = \text{“knowing”}$ with a few segmentation candidates $S(X) = \{\text{‘k, now, ing’}, \text{‘know, ing’}, \text{‘knowing’}\}$. With the unigram probabilities $p(k) = \frac{3}{16}$, $p(now) = \frac{3}{16}$, $p(ing) = \frac{7}{16}$, $p(know) =$

$\frac{3}{16}$ and $p(\text{knowing}) = \frac{3}{16}$ (counts from the word table divided by 16 word occurrences):

$$p(\mathbf{x}_1 = k, \text{now}, \text{ing}) = p(k) p(\text{now}) p(\text{ing}) = \frac{3}{16} \times \frac{3}{16} \times \frac{7}{16} = \frac{63}{4096} \approx 0.0154$$

$$p(\mathbf{x}_2 = \text{know}, \text{ing}) = p(\text{know}) p(\text{ing}) = \frac{3}{16} \times \frac{7}{16} = \frac{21}{256} = \frac{336}{4096} \approx 0.0820$$

$$p(\mathbf{x}_3 = \text{knowing}) = p(\text{knowing}) = \frac{3}{16} = \frac{768}{4096} \approx 0.1875$$

$$\mathbf{x}^* = \arg \max_{\mathbf{x} \in S(X)} P(\mathbf{x}) = \mathbf{x}_3 = \text{knowing}$$

Key idea : the unigram model favours the segmentation with the fewest sub-words

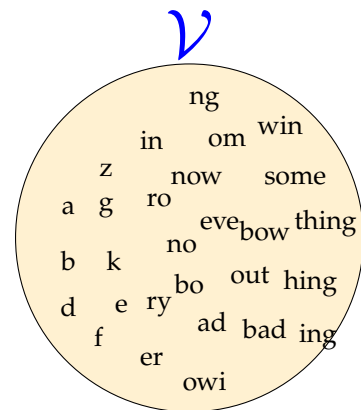
Every factor $P(x_i) < 1$, so each extra token multiplies in another number smaller than one. Unless the shorter pieces are *much* more probable, a segmentation with fewer, longer pieces wins. Frequent whole words stay whole, and rare words get split, which is exactly the behaviour we want from a sub-word tokenizer.

In practice we use **Viterbi decoding** to find $\mathbf{x}^*$ instead of enumerating all possible segmentations. The number of segmentations grows exponentially: a word of length n has up to 2^{n-1} segmentations.

5.4 The Unigram (SentencePiece) training algorithm

Set the desired vocabulary size.

1. Construct a reasonably **large seed vocabulary** using BPE or the Extended Suffix Array algorithm (e.g. all frequent substrings).
2. **E-step:** estimate the probability of every token in the vocabulary using frequency counts in the training corpus.
3. **M-step:** use the Viterbi algorithm to segment the corpus and return the optimal segments that maximize the (log) likelihood.
4. Compute the likelihood for each new sub-word from the optimal segments.
5. **Shrink** the vocabulary by removing the top $x\%$ of sub-words that have the smallest likelihood.
6. Repeat steps 2–5 until the desired vocabulary size is reached.



Note on the slides : E and M labels

Many textbooks label these the other way round (E-step = compute expected counts from the current segmentations, M-step = re-estimate probabilities). The substance is the same alternation: *segment with current probabilities* $\leftrightarrow$ *re-estimate probabilities from segment counts*. Know the slides' version for the exam, and understand that it is ordinary EM.

Beyond the slides : the precise unigram training loop (Kudo, 2018)

1. **E-step:** for each word X^s , compute the posterior over its segmentations $P(\mathbf{x} \mid X^s) \propto P(\mathbf{x})$ with the forward-backward algorithm on the segmentation lattice. Accumulate *expected* counts $\mathbb{E}[c(x)] = \sum_s \sum_{\mathbf{x}} P(\mathbf{x} \mid X^s) n_x(\mathbf{x})$. (Using only the single Viterbi-best segmentation is the cheaper “hard EM” approximation that the slides describe.)
2. **M-step:** $p(x) = \mathbb{E}[c(x)] / \sum_{x'} \mathbb{E}[c(x')]$.
3. **Pruning:** for each token x , compute the **loss** $\text{loss}_x = \mathcal{L} - \mathcal{L}_{\mathcal{V} \setminus \{x\}}$: how much the corpus likelihood would drop if x were removed (its occurrences re-segmented with other tokens). Remove the $\approx 20\%$ of tokens with the smallest loss, but **never remove single characters**, so everything stays encodable.

The model can also *sample* segmentations from $P(\mathbf{x} \mid X)$ during LM training (“subword regularization”), which makes the model robust to segmentation noise.

5.5 Viterbi decoding: segmenting “whereby”

We segment the word “whereby” using the following unigram log-probabilities. Any substring not listed has $\log p = -\infty$ (it is not in the vocabulary).

Token	$\log p(x)$	Token	$\log p(x)$	Token	$\log p(x)$
b	-4.70	wh	-5.99	he	-6.02
e	-2.70	er	-5.34	ere	-6.83
h	-3.34	where	-8.21	here	-7.84
r	-3.36	by	-7.34	her	-7.38
w	-4.29			re	-6.13

The forward algorithm (dynamic programming)

Let $\alpha(t)$ be the best log-probability of any segmentation of the first t characters, $X[1..t]$. Then

Formula : Viterbi recurrence for segmentation

$$\alpha(0) = 0, \quad \alpha(t) = \max_{\substack{0 \leq s < t \\ X[s+1..t] \in \mathcal{V}}} \left[\alpha(s) + \log p(X[s+1..t]) \right], \quad \text{bp}(t) = \arg \max_s [\dots].$$

The answer is $\alpha(n)$. Following the back-pointers bp from $t = n$ to 0 recovers $\mathbf{x}^*$. The cost is $O(n \cdot L_{\max})$, where $L_{\max}$ is the longest token length. Compare that with $O(2^{n-1})$ for enumeration.

Intuition : why this is correct (optimal substructure)

Whatever the best segmentation of “whereby” is, its last piece ends at position 7 and starts at some s . The part before s must itself be the *best* segmentation of the prefix, because otherwise we could swap in a better prefix and improve the total. So we only ever need to remember the best score for each prefix length. This is the same principle as shortest-path algorithms: Viterbi here is a longest path (max log-prob) in a DAG whose nodes are character boundaries.

Iterate over every position in the word and keep the best segmentation of each prefix:

t	Prefix	Candidates $\alpha(s) + \log p(\text{last piece})$	$\alpha(t)$	Best
1	w	w: 0 – 4.29	–4.29	w
2	wh	(w,h): $-4.29 - 3.34 = -7.63$; (wh): -5.99	–5.99	wh
3	whe	(w,he): $-4.29 - 6.02 = -10.31$; (wh,e): $-5.99 - 2.70 = -8.69$; (whe): $-\infty$	–8.69	wh,e
4	wher	(w,her): -11.67 ; (wh,er): $-5.99 - 5.34 = -11.33$; (wh,e,r): $-8.69 - 3.36 = -12.05$; (wher): $-\infty$	–11.33	wh,er
5	where	(where): -8.21 ; (w,here): $-4.29 - 7.84 = -12.13$; (wh,ere): -12.82 ; (α_3 ,re): $-8.69 - 6.13 = -14.82$; (α_4 ,e): $-11.33 - 2.70 = -14.03$	–8.21	where
6	whereb	(α_5 ,b): $-8.21 - 4.70 = -12.91$; (whereb): $-\infty$	–12.91	where,b
7	whereby	(α_5 ,by): $-8.21 - 7.34 = -15.55$; (α_6 ,y): $-\infty$ (no ‘y’)	–15.55	where,by

Backtrack: $\text{bp}(7) = 5$ (last piece “by”), then $\text{bp}(5) = 0$ (piece “where”). The best segmentation of “whereby” that maximizes the likelihood is **“where, by”**.

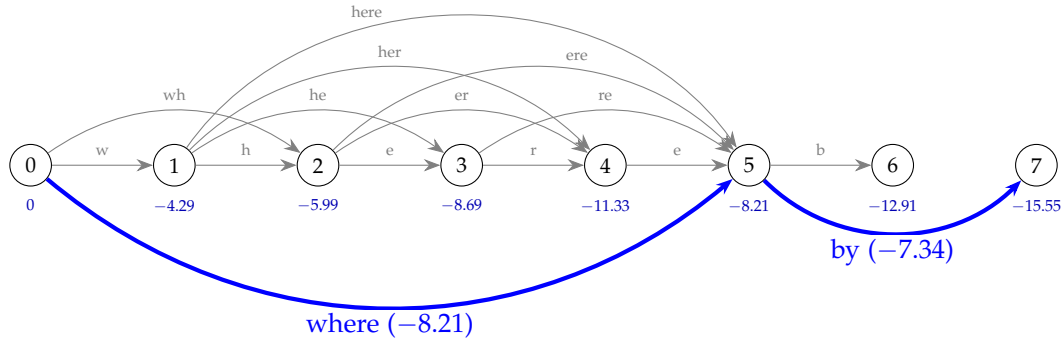


Figure 12: The segmentation lattice for “whereby”. Nodes are character boundaries, each labelled with its best score $\alpha(t)$. Every arc is a vocabulary token. Viterbi finds the highest-scoring path, shown in blue: *where, by*.

Pruning in the slides’ tree view. The slides draw the same computation as a tree and cross out dominated branches. For example, once (wh) beats (w,h) for the prefix “wh” ($-5.99 > -7.63$), we never need to extend (w,h), so “we do not need to compute the likelihood of (w,h,e) as we already ruled out (w,h) in favour of (wh)”. Of the options for “whe”, (wh,e) is the best. For “where”, the single token where (-8.21) beats every multi-piece option.

Note on the slides : two small arithmetic slips

(1) The slides label the branch (w, here) as -10.54 . The correct value is $-4.29 - 7.84 = -12.13$. It is pruned either way, since where at -8.21 is better. (2) The final score is shown as -15.51 . The correct value is $-8.21 - 7.34 = -15.55$. The conclusion, **where, by**, is unaffected.

We can follow the same procedure for languages that do not use any word delimiters: just run Viterbi over the whole sentence’s character string.

Systems / engineering note : Viterbi in practice

SentencePiece stores the vocabulary in a double-array *trie*, so at each position all vocabulary tokens that start there are found in one pass over at most $L_{\max}$ characters. Encoding is then linear in the input length. Because this is exact optimisation, the unigram tokenizer has no “merge order” to store: the model file is just (piece, log-prob) pairs.

5.6 Morfessor

Recall that the unigram model uses **Maximum Likelihood Estimation (MLE)** as its objective. Another algorithm related to SentencePiece is **Morfessor**, which uses **Maximum a Posteriori (MAP)** estimation, and that lets us incorporate **a prior**.

Formula : MLE vs MAP

$$\theta_{\text{MLE}} = \arg \max_{\theta} P(D \mid \theta), \quad \theta_{\text{MAP}} = \arg \max_{\theta} P(\theta \mid D) = \arg \max_{\theta} \underbrace{P(D \mid \theta)}_{\text{likelihood}} \underbrace{P(\theta)}_{\text{prior}}.$$

In log form: $\arg \min_{\theta} [-\log P(D \mid \theta) - \log P(\theta)]$. Morfessor's prior prefers a *compact lexicon* of morphs, which is the Minimum Description Length (MDL) principle: total cost = cost of the lexicon + cost of the data encoded with it.

Worked example : *Sesquipedalophobia* (fear of long words)

Suppose SentencePiece returns one best segmentation: **Ses-qui-ped-alo-phobia**. For some reason (say, linguistic correctness) we want it to be **Ses-qui-pedalo-phobia**, because “pedalo” is the morph (Latin *pes, pedis* “foot”). In general, Morfessor lets us incorporate linguistically motivated **a priori** knowledge into the tokenization procedure.

- Morfessor also lets us add a **small annotated dataset** while training the model (semi-supervised): the training data and optional annotation data both go into `morfessor-train`, which outputs a Morfessor model.
- It has been used for **large-vocabulary continuous speech recognition**, especially for morphologically rich languages such as Finnish, where word-level vocabularies explode.

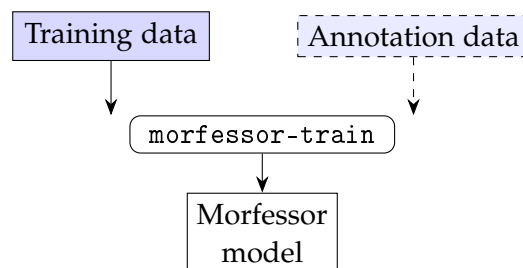


Figure 13: Morfessor training can combine unannotated data with a small amount of annotated segmentations.

6 Putting it all together

	BPE	WordPiece	Unigram (SentencePiece)
Direction	Bottom-up: merge	Bottom-up: merge	Top-down: prune a large seed vocab
Criterion	Pair frequency	$\frac{c(\alpha\beta)}{c(\alpha)c(\beta)}$ (PMI / likelihood gain)	Corpus likelihood under unigram LM (EM)
Output per word	One, deterministic (greedy)	One, deterministic (longest match)	Best via Viterbi (or sampled)
Needs pre-tokenized words	Yes (classic)	Yes	No: treats whitespace as a symbol
Probabilistic	No	Partly (in training)	Yes
Stores	Merge list	Vocabulary	Vocab + log-probs
Examples	GPT-2/3/4, RoBERTa, Llama 3	BERT, DistilBERT	T5, ALBERT, XLNet, mBART

Common pitfall / exam trap : “SentencePiece” vs “Unigram”

SentencePiece is a library/framework (raw-text input, whitespace as `_`, language agnostic). It supports *both* a unigram-LM model *and* a BPE model. For example, Llama 1/2 used SentencePiece with BPE. The lecture’s Module 4.4 describes the unigram-LM algorithm, which is SentencePiece’s default.

References (from the slides)

- Sennrich, Haddow, Birch (2016). *Neural Machine Translation of Rare Words with Subword Units*.
- Kudo (2018). *Subword Regularization* (unigram LM); Kudo & Richardson (2018). *SentencePiece*.
- Rust implementation of SentencePiece by Guillaume.
- Kyle Chung, *On Subword Units* (blog).
- Morfessor (Creutz & Lagus; Virpioja et al.).

7 Quick-revision cheat sheet

Concept	Must-remember fact
Pipeline	text $\rightarrow$ tokenizer $\rightarrow$ ids $\rightarrow$ embeddings $\rightarrow$ LM $\rightarrow$ ids $\rightarrow$ tokens $\rightarrow$ text.
Vocab challenges	size (embedding + softmax cost), OOV, misspellings, open vocabulary (agglutinative languages, names, numbers).
Granularities	character (tiny $\mathcal{V}$, long sequences), word (huge $\mathcal{V}$, OOV), sub-word (moderate $\mathcal{V}$; frequent words kept whole, rare words split).
Wishlist	moderate $\mathcal{V}$; handle unknown words; language agnostic.
Pre-processing	normalization $\rightarrow$ pre-tokenization $\rightarrow$ sub-word training.
BPE	add $\langle \text{w} \rangle$; start from characters; repeatedly merge the most frequent adjacent pair; $ \mathcal{V} = \mathcal{V}_0 + \text{\#merges}$; inference replays merges in order.
BPE limits	needs space-separated words (Japanese needs Juman++/MeCab); greedy, deterministic; ties arbitrary.
WordPiece	score = $\frac{c(\alpha, \beta)}{c(\alpha)c(\beta)}$ (PMI-like); prefers pairs of rare parts; ## continuation prefix.
Growing vs shrinking	BPE/WordPiece: merge upward from characters. Unigram: prune downward from a large seed.
Unigram LM	$P(\mathbf{x}) = \prod_i P(x_i)$; $\mathbf{x}^* = \arg \max_{\mathbf{x} \in \mathcal{S}(X)} P(\mathbf{x})$; favours fewer sub-words.
Corpus likelihood	$\mathcal{L} = \sum_s \log \sum_{\mathbf{x} \in \mathcal{S}(X^s)} P(\mathbf{x})$; latent segmentation, so EM.
Training loop	seed vocab $\rightarrow$ estimate $p \rightarrow$ Viterbi segment $\rightarrow$ likelihood per sub-word $\rightarrow$ drop lowest $x\%$ $\rightarrow$ repeat.
Viterbi	$\alpha(t) = \max_s [\alpha(s) + \log p(X_{s+1..t})]$, then backtrack; “whereby” $\rightarrow$ where, by.
Morfessor	MAP (likelihood $\times$ prior) instead of MLE; can use annotated data; used in LVCSR.

8 Numerical problems with worked solutions

Problem 1. Cost of the vocabulary

A model has $d = 1024$. Compare the parameters of the input embedding matrix for $|\mathcal{V}| = 32,000$ (sub-word) and $|\mathcal{V}| = 500,000$ (word level). If input and output embeddings are *not* tied, what is the total for each? What fraction of a 350M-parameter model is the embedding in each case?

Solution

Sub-word: $32,000 \times 1024 = 32.8\text{M}$ per matrix, 65.5M untied. Word: $500,000 \times 1024 = 512\text{M}$ per matrix, 1.024B untied. Fraction of 350M: the sub-word model spends $65.5/350 \approx 19\%$ on embeddings. For the word-level model the embeddings alone (1.02B) are *larger than the whole intended model*. The output softmax also costs $2|\mathcal{V}|d$ FLOPs per token: 65.5M vs 1.02G per position. This is the first “vocabulary challenge” in numbers.

Problem 2. Counting tokens at three granularities

For the text “I don’t know”, give the number of tokens and the vocabulary for (a) character level, (b) whitespace word level, (c) a sub-word tokenizer with $\mathcal{V} = \{I, know, do, n't\}$.

Solution

(a) Characters: I, $_$, d, o, n, ', t, $_$, k, n, o, w, so **12 tokens**. Distinct characters $\{I, _, d, o, n, ', t, k, w\}$: $|\mathcal{V}| = 9$. (b) Words: [I, don’t, know], so **3 tokens**, $|\mathcal{V}| = 3$. (c) Sub-words: [I, do, n’t, know], so **4 tokens**, $|\mathcal{V}| = 4$. “n’t” is reusable for *can’t*, *isn’t*, and so on.

Problem 3. Character counts from a word table

Using the lecture corpus word table, verify that $c('n') = 13$, $c('i') = 10$, $c('t') = 8$ and $c('</w>') = 16$.

Solution

'n': knowing $(2 \times 3) + \text{name } 1 + \text{something } (1 \times 2) + \text{different } 1 + \text{something. } 1 + \text{everything } 1 + \text{isn't } 1 = 13$. 'i': knowing $(1 \times 3) + \text{something } 2 + \text{is } 1 + \text{different } 1 + \text{something. } 1 + \text{everything } 1 + \text{isn't } 1 = 10$. 't': the $1 + \text{something } 2 + \text{different } 1 + \text{something. } 1 + \text{about } 1 + \text{everything } 1 + \text{isn't } 1 = 8$. '</w>': one per word occurrence = $3 + 1 + 1 + 1 + 2 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 = 16$.

Problem 4. Full BPE run on a new corpus

Word counts: {hug: 10, pug: 5, pun: 12, bun: 4, hugs: 5}. Append </w>. (a) Give the initial vocabulary and its size. (b) List all initial pair counts. (c) Perform 4 merges, showing the corpus after each. (d) How many symbols does the corpus have before and after?

Solution

(a) Characters: u (36), </w> (36), g (20), p (17), n (16), h (15), s (5), b (4), so $|\mathcal{V}_0| = 8$. The total is 149 symbols.

(b) $(u,g) = 10 + 5 + 5 = 20$; $(p,u) = 5 + 12 = 17$; $(u,n) = 12 + 4 = 16$; $(n,</w>) = 16$; $(h,u) = 10 + 5 = 15$; $(g,</w>) = 10 + 5 = 15$; $(g,s) = 5$; $(s,</w>) = 5$; $(b,u) = 4$.

(c) **Merge 1: $(u,g) \rightarrow ug$ (20).** Corpus: h ug </w>:10, p ug </w>:5, p u n </w>:12, b u n </w>:4, h ug s </w>:5.

Updated pairs: $(u,n)=16$, $(n,</w>)=16$, $(h,ug)=15$, $(ug,</w>)=15$, $(p,u)=12$, ...

Merge 2: $(u,n) \rightarrow un$ (16, tie with $(n,</w>)$; first found wins). Corpus: ... p un </w>:12, b un </w>:4.

Merge 3: $(un,</w>) \rightarrow un</w>$ (16).

Merge 4: $(h,ug) \rightarrow hug$ (15, tie with $(ug,</w>)$). Corpus: hug </w>:10, p ug </w>:5, p un</w>:12, b un</w>:4, hug s </w>:5.

(d) Before: 149. After each merge the corpus shrinks by the merge count: $149 - 20 = 129$, $-16 = 113$, $-16 = 97$, $-15 = 82$. The vocabulary is now $8 + 4 = 12$.

Problem 5. BPE inference with learned merges

Using the 4 merges from the previous problem, in order [(u,g), (u,n), (un,</w>), (h,ug)], encode *bug*, *hugs*, *puns* and *mug*.

Solution

bug: b u g </w> $\rightarrow$ (u,g) $\rightarrow$ b ug </w>. No further merge applies: [b, ug, </w>].

hugs: h u g s </w> $\rightarrow$ h ug s </w> $\rightarrow$ (h,ug) $\rightarrow$ [hug, s, </w>].

puns: p u n s </w> $\rightarrow$ (u,n) $\rightarrow$ p un s </w>. (un,</w>) does not apply because 's' sits in between: [p, un, s, </w>].

mug: 'm' is not in the base vocabulary, so it becomes [<UNK>, ug, </w>]. With *byte-level* BPE this could never happen, because every byte is in the base vocabulary.

Problem 6. Lecture corpus: effect of the first merge

In the lecture corpus, after merging (i,n), give the counts of 'i', 'n' and 'in', and the new counts of the pairs (w,in), (in,g) and (h,in). Which pair is merged next?

Solution

'in' = 7; 'n' = 13 - 7 = 6; 'i' = 10 - 7 = 3 (remaining i's: is, different, isn't). (w,in) = 3 (knowing $\times 3$); (in,g) = 7; (h,in) = 2 + 1 + 1 = 4 (something $\times 2$, something., everything). The pair (g,</w>) is still 6, since the g's at word ends are untouched. The next merge is (in,g) $\rightarrow$ ing with count 7.

Problem 7. Vocabulary size vs merges

A character-level base vocabulary (after normalization) has 112 symbols. How many merges are needed for a vocabulary of 32,000? With byte-level BPE (256 bytes) and 1 special token, how many merges give GPT-2's 50,257?

Solution

$32,000 - 112 = 31,888$ merges. GPT-2: $50,257 - 256 - 1 = 50,000$ merges.

Problem 8. WordPiece scores

For the corpus {hug:10, pug:5, pun:12, bun:4, hugs:5} (no </w>, prefix ignored), compute the WordPiece score of every pair. Which pair does WordPiece merge first? Compare with BPE.

Solution

Character counts: u 36, g 20, p 17, n 16, h 15, s 5, b 4.

Pair	$c(\alpha, \beta)$	$c(\alpha)c(\beta)$	score
(u,g)	20	$36 \cdot 20 = 720$	0.0278
(p,u)	17	$17 \cdot 36 = 612$	0.0278
(u,n)	16	$36 \cdot 16 = 576$	0.0278
(h,u)	15	$15 \cdot 36 = 540$	0.0278
(b,u)	4	$4 \cdot 36 = 144$	0.0278
(g,s)	5	$20 \cdot 5 = 100$	0.05

WordPiece merges (g,s) first, a pair BPE would merge almost last (count 5). BPE merges (u,g) first (count 20). All the 'u' pairs score exactly $1/36$: every pair (x, u) with $c(x, u) = c(x)$ has score $1/c(u)$. WordPiece sees "s almost always follows g", so the pair is "sticky", while 'u' pairs are unremarkable because u is everywhere.

Problem 9. *Lecture corpus: WordPiece vs BPE on the tie*

BPE faced a tie between (i,n) and (n,g) at count 7. Compute both WordPiece scores and state which is preferred and why.

Solution

(i,n): $7/(10 \cdot 13) = 0.0538$. (n,g): $7/(13 \cdot 7) = 0.0769$. WordPiece prefers **(n,g)**, because 'g' is rarer than 'i' (7 vs 10) and every 'g' follows an 'n' (the pair explains *all* occurrences of g).

Problem 10. *Unigram segmentation probabilities*

With $p(\text{un}) = 0.05$, $p(\text{happy}) = 0.02$, $p(\text{hap}) = 0.01$, $p(\text{py}) = 0.03$, $p(u) = p(n) = 0.04$, compare the segmentations of *unhappy*: (a) un, happy; (b) un, hap, py; (c) u, n, happy. Which is chosen?

Solution

(a) $0.05 \times 0.02 = 1.0 \times 10^{-3}$. (b) $0.05 \times 0.01 \times 0.03 = 1.5 \times 10^{-5}$. (c) $0.04 \times 0.04 \times 0.02 = 3.2 \times 10^{-5}$. **(a) un, happy** wins by a factor of 30 or more, with the fewest sub-words. Note that (c) has more tokens than (a) but beats (b), because its extra pieces are relatively probable. "Fewest pieces" is a strong tendency, not a hard law.

Problem 11. *Viterbi on a new word*

Segment *unhappy* with Viterbi, using $\log p$: u -5.0 , n -3.5 , h -3.8 , a -3.0 , p -4.0 , y -4.2 , un -4.5 , ha -5.5 , ap -5.8 , hap -7.0 , app -6.5 , py -6.0 , happ -9.5 , ppy -7.5 , happy -8.0 , unh -9.0 .

Solution

t	prefix	candidates	$\alpha(t)$	best
1	u	u: -5.0	-5.0	u
2	un	(un) -4.5 ; (u,n) -8.5	-4.5	un
3	unh	(α_2 ,h) -8.3 ; (unh) -9.0	-8.3	un,h
4	unha	(α_2 ,ha) -10.0 ; (α_3 ,a) -11.3	-10.0	un,ha
5	unhap	(α_2 ,hap) -11.5 ; (α_4 ,p) -14.0 ; (α_3 ,ap) -14.1	-11.5	un,hap
6	unhapp	(α_2 ,happ) -14.0 ; (α_3 ,app) -14.8 ; (α_5 ,p) -15.5	-14.0	un,happ
7	unhappy	(α_2 ,happy) -12.5 ; (α_5 ,py) -17.5 ; (α_4 ,ppy) -17.5 ; (α_6 ,y) -18.2	-12.5	un, happy

Backtrack: $\text{bp}(7)=2$ ("happy"), $\text{bp}(2)=0$ ("un"), so **[un, happy]**, $\log P = -12.5$. This is also the morphologically correct split. Note that $\alpha(6)$'s best path (un, happ) is *not* a prefix of the final answer: Viterbi keeps every prefix's best score, not a single greedy path.

Problem 12. *Counting segmentations*

(a) List all segmentations of *hello* with $\mathcal{V} = \{h, e, l, o, he, el, lo, ll, hell\}$. (b) If *every* substring were in the vocabulary, how many segmentations would a word of length n have? (c) How many for "whereby"?

Solution

(a) 9 segmentations: [h,e,l,l,o], [h,e,l,lo], [h,e,ll,o], [h,el,l,o], [h,el,lo], [he,l,l,o], [he,l,lo], [he,ll,o], [hell,o].

(b) Between each of the $n - 1$ adjacent character pairs we choose "cut" or "no cut", so 2^{n-1} .

(c) $2^6 = 64$ at most (fewer with the actual vocabulary). Enumeration is exponential, while Viterbi is $O(n L_{\max})$.

Problem 13. *One round of (hard) EM for the unigram model*

Corpus {ab: 6, abc: 3, bc: 2, c: 1}. Vocabulary with initial probabilities $p(a) = 0.25$, $p(b) = 0.15$, $p(c) = 0.2$, $p(ab) = 0.2$, $p(bc) = 0.2$. (a) Find the Viterbi segmentation of each word. (b) Re-estimate p from the segment counts. (c) Compute the (Viterbi) corpus log-likelihood before and after. (d) Which token would be pruned first?

Solution

(a) ab: [ab] 0.2 vs [a,b] 0.0375, so **[ab]**. abc: [a,bc] 0.05, [ab,c] 0.04, [a,b,c] 0.0075, so **[a,bc]**. bc: [bc] 0.2 vs [b,c] 0.03, so **[bc]**. c: [c].
 (b) Counts: ab 6; a 3, bc 3 (from abc); bc +2 (from bc), so bc 5; c 1; b 0. Total 15. New p : $a = 0.2$, $b = 0$, $c = 0.0667$, $ab = 0.4$, $bc = 0.3333$.
 (c) Before: $6 \ln 0.2 + 3 \ln 0.05 + 2 \ln 0.2 + \ln 0.2 = -23.47$. After re-estimation (same segmentations, new p): $6 \ln 0.4 + 3 \ln(0.2 \cdot 0.333) + 2 \ln 0.333 + \ln 0.0667 = -18.53$. The likelihood increased, as EM guarantees. A further iteration changes nothing (converged).
 (d) 'b' has zero probability (zero likelihood contribution), so it is the natural first to remove. In real SentencePiece, however, single characters are never pruned, so that every string remains encodable.

Problem 14. *WordPiece longest-match encoding*

Vocabulary: {un, ##aff, ##able, aff, ##a, ##ff, u, ##n, ##b, ##l, ##e}. Encode *unaffable*. What happens to *xyz*?

Solution

Start: longest prefix in $\mathcal{V}$ of “unaffable” is un. Remaining “affable” needs ##-pieces: the longest is ##aff. Remaining “able”: ##able. Result: **un ##aff ##able**. For *xyz*: no piece matches ‘x’, so the whole word becomes [UNK].

Problem 15. *Sequence length and attention cost*

An English sentence has 20 words, 110 characters (with spaces) and 26 sub-word tokens. Compare the self-attention score-matrix sizes (T^2 per head) for the three tokenizations.

Solution

Word: $20^2 = 400$. Sub-word: $26^2 = 676$ ($1.7 \times$ word). Character: $110^2 = 12,100$, which is $\approx 17.9 \times$ sub-word and $30 \times$ word. That is why character-level transformers are rarely used at scale despite having no OOV problem.

Problem 16. *Fertility and cost*

A tokenizer averages 1.3 tokens per English word and 3.9 tokens per Hindi word. A model has a 4,096-token context and costs \$1 per million tokens. How many words of each language fit in the context, and what does 1M words cost?

Solution

English: $4096/1.3 \approx 3151$ words; 1M words = 1.3M tokens, which is \$1.30. Hindi: $4096/3.9 \approx 1050$ words; 1M words = 3.9M tokens, which is \$3.90. Hindi is $3 \times$ more expensive with a $3 \times$ smaller effective context. This is a strong argument for training tokenizers on multilingual data (and for Indic tokenizers such as those built by AI4Bharat).

9 Subjective questions with model answers

Problem 1.

Why is whitespace splitting not a good tokenization strategy for LLMs?

Answer

(1) It creates huge vocabularies (every inflection, name, number and typo becomes a word), inflating the embedding matrix and softmax. (2) Unseen words at test time become OOV (<UNK>), losing information. (3) Related forms (enjoy/enjoyed) get unrelated ids, so there is no parameter sharing. (4) Punctuation sticks to words ("know," $\neq$ "know"). (5) It does not work for languages without spaces (Japanese, Chinese, Thai). (6) Agglutinative languages create unboundedly many word forms (open vocabulary).

Problem 2.

Compare character-level, word-level and sub-word tokenization.

Answer

Character: tiny vocabulary, no OOV, robust to typos; but very long sequences (quadratic attention cost, context used up quickly), and each unit carries little meaning. **Word:** short sequences of meaningful units; but huge vocabulary, OOV, no sharing between related forms, and it requires word boundaries. **Sub-word:** a middle ground. The vocabulary is built from subword frequency. Frequent words stay whole and rare words split into known pieces ("don't" $\rightarrow$ "do" + "n't"). The vocabulary is moderate, there is virtually no OOV, and pieces are shared. Hence it is preferred for LLMs.

Problem 3.

Explain the BPE training algorithm, and why the end-of-word symbol </w> is needed.

Answer

Pre-tokenize the corpus into words with counts, split each word into characters, and append </w>. The base vocabulary is the set of characters. Repeat for the chosen number of merges: count all adjacent symbol pairs weighted by word frequency, merge the most frequent pair into a new symbol everywhere, and add it to the vocabulary (recording the merge order). The final vocabulary is the base plus all merges. </w> marks word boundaries. It lets BPE distinguish a suffix at the end of a word ("est</w>" in "widest") from the same letters in the middle of a word ("est" in "estimate"), and it lets detokenization know where to insert spaces.

Problem 4.

How does a trained BPE tokenizer handle a word never seen in training?

Answer

The word is split into characters (plus </w>), and the learned merges are applied in the order they were learned, fusing adjacent pairs whenever a merge rule matches. The result is a sequence of known sub-words. For instance "everything" might become "everyth + ing". In the worst case, if no merges apply, it falls back to single characters, which are always in the base vocabulary. So there is no <UNK>, unless a character itself was never seen, which byte-level BPE eliminates by using the 256 bytes as the base.

Problem 5.

Why does the number of merges act as a hyperparameter? What happens at the extremes?

Answer

Each merge adds exactly one token, so $|\mathcal{V}| = |\mathcal{V}_0| + N$. With $N = 0$ we have a character-level tokenizer: long sequences, tiny vocabulary. With $N \rightarrow \text{“all possible”}$, every training word becomes a single token and we have effectively a word-level tokenizer with its big-vocabulary and OOV problems. Choosing N trades sequence length against vocabulary size. Typical values give 30K–50K (monolingual) or 100K–250K (multilingual) tokens.

Problem 6.

Why is BPE problematic for Japanese, and what are the two ways around it?

Answer

BPE’s training starts from a dictionary of *words* produced by whitespace pre-tokenization. Japanese has no spaces, so the entire sentence would be one “word”. Fix 1: use a language-specific morphological segmenter (Juman++, MeCab) as the pre-tokenizer, then run BPE. This works, but it is language-dependent and does not scale to multilingual or code-mixed text. Fix 2: use a language-agnostic method such as SentencePiece, which treats text as a raw character stream (whitespace as an ordinary symbol) and learns segmentations without pre-tokenization.

Problem 7.

Derive and interpret the WordPiece score. How does it differ from BPE’s criterion?

Answer

$\text{score}(\alpha, \beta) = \frac{c(\alpha, \beta)}{c(\alpha)c(\beta)}$. With N total symbols, $\frac{P(\alpha\beta)}{P(\alpha)P(\beta)} = N \cdot \text{score}$, so the ranking is by pointwise mutual information: how much more often α and β appear together than if they were independent. It also approximates the increase in unigram likelihood from merging. BPE ranks by raw $c(\alpha, \beta)$, which favours pairs of frequent symbols that co-occur often by chance. WordPiece favours pairs whose parts are individually rare but consistently adjacent, and it naturally breaks BPE’s frequency ties (in the lecture, (n,g) 0.077 beats (i,n) 0.054).

Problem 8.

“BPE is greedy and deterministic.” Explain with an example, and explain how the probabilistic approach differs.

Answer

With $\mathcal{V} = \{h, e, l, o, he, el, lo, ll, hell\}$, “hello” has 9 valid segmentations. BPE always outputs the one produced by replaying its merges in order: he,l,lo if he was learned before e1; h,e,l,lo if the other way round. The choice depends on merge order, not on any global quality measure, and is always the same (deterministic). The probabilistic approach treats the segmentation as a hidden variable, assigns each candidate a probability under a unigram LM ($P(\mathbf{x}) = \prod P(x_i)$), and picks $\mathbf{x}^* = \arg \max P(\mathbf{x})$ via Viterbi. It can also *sample* segmentations (subword regularization).

Problem 9.

Why is EM needed to train the unigram tokenizer?

Answer

The corpus likelihood is $\mathcal{L} = \sum_s \log \sum_{\mathbf{x} \in S(X^s)} \prod_i P(x_i)$. The segmentation of each word is latent, so there is a sum inside the log and no closed-form MLE. If the segmentations were known, the MLE would be simple counting; if the probabilities were known, the best segmentation follows from Viterbi. EM alternates these two steps: segment (or compute expected counts) with the current probabilities, then re-estimate probabilities from the counts. Each iteration does not decrease $\mathcal{L}$.

Problem 10.

Describe the unigram vocabulary-learning procedure step by step. How does it differ in direction from BPE?

Answer

Fix a target vocabulary size. (1) Build a large seed vocabulary (e.g. with BPE or the Extended Suffix Array: frequent substrings). (2) Estimate each token's probability from frequency counts. (3) Viterbi-segment the corpus into optimal segments. (4) Compute each sub-word's likelihood contribution from the optimal segments. (5) Remove the $x\%$ of sub-words with the smallest likelihood (how much the likelihood would drop without them). (6) Repeat 2–5 until the target size is reached. BPE and WordPiece grow a vocabulary upward from characters by merging. Unigram shrinks a large vocabulary downward by elimination.

Problem 11.

Explain Viterbi decoding for segmentation using “whereby”.

Answer

Define $\alpha(t)$ as the best log-probability of segmenting the first t characters, with $\alpha(t) = \max_s [\alpha(s) + \log p(X_{s+1..t})]$ over vocabulary tokens ending at t . For “whereby”: $\alpha(1) = -4.29$ (w), $\alpha(2) = -5.99$ (wh beats w,h at -7.63), $\alpha(3) = -8.69$ (wh,e), $\alpha(4) = -11.33$ (wh,er), $\alpha(5) = -8.21$ (the single token “where” beats all others), $\alpha(6) = -12.91$ (where,b), $\alpha(7) = -15.55$ (where,by). Backtracking gives **where, by**. Dominated partial segmentations never need to be extended, which gives $O(nL_{\max})$ time instead of exponential enumeration. The same procedure works for whole sentences in languages without spaces.

Problem 12.

Why does the unigram LM favour segmentations with fewer sub-words? Is that always true?

Answer

$P(\mathbf{x})$ is a product of probabilities, each < 1 , so each extra factor tends to shrink the product. In the lecture example, “knowing” as one token has $P = 3/16 = 0.1875$, while know+ing has 0.082 and k+now+ing has 0.015. It is not a strict rule: a segmentation with more but much more probable pieces can win (see numerical Problem 10, where (c) beats (b)). The preference emerges from likelihood, not from a hard-coded length penalty.

Problem 13.

What is Morfessor and how does it differ from the unigram model used in SentencePiece?

Answer

Both model words as sequences of sub-word units (morphs). The unigram model is trained by MLE: maximize $P(D | \theta)$. Morfessor uses MAP estimation, maximizing $P(D | \theta)P(\theta)$. The prior (typically favouring a compact lexicon, in MDL style) lets us inject linguistic a priori knowledge, and Morfessor can also use a small annotated dataset of correct segmentations (semi-supervised). Example: prefer ses-qui-pedalo-phobia over the likelihood-optimal ses-qui-ped-alo-phobia. It has been used for large-vocabulary continuous speech recognition.

Problem 14.

Evaluate BPE, WordPiece and SentencePiece against the wishlist (moderate vocabulary, unknown words, language agnostic).

Answer

Moderate vocabulary: all three, since the size is directly controlled (merges or pruning). **Unknown words:** all three decompose unseen words into known sub-words. BPE falls back to characters (byte-level BPE guarantees coverage), WordPiece may emit [UNK] if a character is missing, and Unigram keeps all characters. **Language agnostic:** classic BPE and WordPiece need whitespace pre-tokenization, so they need a segmenter for Japanese and Chinese. SentencePiece works on raw text with whitespace as a symbol, so it is language agnostic. It is the best fit for multilingual models.

10 Multiple-choice questions

Q1. The step that splits text into words before a sub-word algorithm is called

- (A) normalization (B) pre-tokenization
- (C) post-processing (D) detokenization

Answer: B

When followed by a sub-word algorithm, whitespace splitting is called pre-tokenization.

Q2. Lower-casing and removing accents belong to which stage?

- (A) Normalization (B) Pre-tokenization
- (C) Merging (D) Viterbi decoding

Answer: A

Normalization treats case, accents, multiple whitespaces and HTML tags.

Q3. Which is *not* one of the listed challenges of building a vocabulary?

- (A) vocabulary size
- (B) out-of-vocabulary words
- (C) misspelled words
- (D) vanishing gradients

Answer: D

The lecture lists size, OOV, misspellings and the open-vocabulary problem.

Q4. Increasing $|\mathcal{V}|$ directly increases

- (A) the number of transformer layers
- (B) the size of the embedding matrix and the softmax cost
- (C) the attention head dimension
- (D) the context length

Answer: B

Embedding is $|\mathcal{V}| \times d$, and the softmax is over $|\mathcal{V}|$ classes.

Q5. In sub-word tokenization, which words are typically kept whole?

- (A) Rare words (B) Frequent words
- (C) Long words (D) Misspelled words

Answer: B

Frequent words are preserved; rare words are split into sub-word units.

Q6. The sub-word tokenizer splits “don’t” into “do” and “n’t” because

- (A) “n’t” occurs frequently in the corpus
- (B) apostrophes are illegal
- (C) “don’t” is longer than 4 characters
- (D) of a hand-written rule

Answer: A

It learns that “n’t” is a frequent unit.

Q7. What is the role of $\langle /w \rangle$ in BPE?

- (A) marks the end of a sentence
- (B) marks the word boundary
- (C) a padding token
- (D) the unknown token

Answer: B

It identifies the word boundary, distinguishing word-final from word-internal pieces.

Q8. In the lecture corpus, the initial vocabulary size (characters + $\langle /w \rangle$) is

- (A) 13 (B) 16
- (C) 22 (D) 45

Answer: C

21 distinct characters plus $\langle /w \rangle$. (13 is the number of distinct words; 16 is the count of $\langle /w \rangle$.)

Q9. In the lecture corpus, the count of the pair (i,n) is

- (A) 3 (B) 4
(C) 7 (D) 10

Answer: C

knowing 3 + something 2 + something. 1 + everything 1.

Q10. After merging (i,n) in the lecture corpus, the count of the token 'n' becomes

- (A) 13 (B) 7
(C) 6 (D) 3

Answer: C

$13 - 7 = 6$.

Q11. After the first merge, the pair (h,in) has count

- (A) 3 (B) 4
(C) 7 (D) 2

Answer: B

something (2) + something. (1) + everything (1).

Q12. In BPE, the final vocabulary size equals

- (A) the number of unique words
(B) base vocabulary size + number of merges
(C) number of merges only
(D) always 30,000

Answer: B

Each merge adds exactly one new token.

Q13. At inference time, a trained BPE tokenizer segments a word by

- (A) longest-match over the vocabulary
(B) splitting into characters and applying merges in learned order
(C) Viterbi decoding
(D) random sampling

Answer: B

Replay the merges in the order they were learned.

Q14. Given $\mathcal{V} = \{a, \dots, z, lo, he\}$ with merge order lo then he, "hello lol" is tokenized as

- (A) hel lo, lol
(B) he #l #lo, lo #l
(C) h #e #l #l #o, l #o #l
(D) hello, lol

Answer: B

Merge 'lo' then 'he': [he l lo], [lo l].

Q15. Why can't BPE be directly applied to Japanese?

- (A) Japanese has too many characters
- (B) BPE requires space-separated words
- (C) Japanese is right-to-left
- (D) BPE only works on ASCII

Answer: B

Japanese is non-segmented; BPE needs whitespace pre-tokenization (or a segmenter like Juman++/MeCab).

Q16. Juman++ and MeCab are

- (A) sub-word algorithms
- (B) language-specific morphological word segmenters
- (C) byte-level encoders
- (D) LLMs

Answer: B

They are used as pre-tokenizers for Japanese before running BPE.

Q17. The WordPiece merge score is

- (A) $c(\alpha, \beta)$
- (B) $c(\alpha)c(\beta)/c(\alpha, \beta)$
- (C) $c(\alpha, \beta)/(c(\alpha)c(\beta))$
- (D) $c(\alpha) + c(\beta)$

Answer: C

Pair count divided by the product of the individual counts.

Q18. With $c(t, h) = 5$, $c(t) = 8$, $c(h) = 5$, the WordPiece score of (t,h) is

- (A) 0.625 (B) 0.125
- (C) 0.2 (D) 5

Answer: B

$5/(8 \times 5) = 0.125$.

Q19. WordPiece tends to merge pairs whose individual tokens are

- (A) very frequent
- (B) less frequent in the vocabulary
- (C) longer than 3 characters
- (D) punctuation

Q25. Why is EM used to train the unigram model?

- (A) The loss is non-differentiable
- (B) The segmentations (sub-word assignments) are latent variables
- (C) To speed up Viterbi
- (D) Because the vocabulary is fixed

Answer: B

The likelihood sums over hidden segmentations inside the log.

Q26. The Viterbi-best segmentation of “whereby” (lecture table) is

- (A) wh, ere, by (B) w, here, by
- (C) where, by (D) wh, e, r, e, by

Answer: C

$-8.21 - 7.34 = -15.55$, the maximum.

Q27. For the prefix “wh”, Viterbi keeps

- (A) (w,h) with -7.63 (B) (wh) with -5.99
- (C) both (D) neither

Answer: B

$-5.99 > -7.63$; only the best per prefix is kept.

Q28. The time complexity of Viterbi segmentation of a length- n string with max token length L is

- (A) $O(2^n)$ (B) $O(nL)$
- (C) $O(n^3)$ (D) $O(|\mathcal{V}|^n)$

Answer: B

Each position checks at most L previous positions.

Q29. Morfessor differs from the unigram model mainly by using

- (A) MLE
- (B) MAP estimation with a prior
- (C) reinforcement learning
- (D) byte-level merges

Answer: B

MAP lets it incorporate prior (linguistic) knowledge and annotated data.

Q30. SentencePiece is language agnostic mainly because

- (A) it uses a separate segmenter per language
- (B) it treats the input as a raw character stream, with whitespace as a normal symbol
- (C) it only supports English
- (D) it removes all spaces permanently

Answer: B

No pre-tokenization is needed, so it works for Chinese and Japanese.

Q31. Which model family uses WordPiece?

- (A) GPT-2 (B) BERT
(C) T5 (D) Llama 3

Answer: B

BERT's 30,522-token vocabulary is WordPiece. GPT-2 and Llama 3 use byte-level BPE; T5 uses SentencePiece unigram.

Q32. In the vocabulary-size diagram, a “medium” vocabulary typical of monolingual models is

- (A) 1–112 (B) 30K–50K
(C) 100K–250K (D) 1M+

Answer: B

Larger 100K–250K vocabularies are typical of multilingual models.